🔹 Challenge Técnico — Beilox

QA Automation Senior

🎯 Objetivo

Evaluar la capacidad del candidato para desarrollar pruebas automatizadas usando **Playwright con TypeScript**, aplicando buenas prácticas tanto para **UI web como API**, con una arquitectura escalable, uso responsable de asistentes de IA con **MCP servers** (Claude Code, GitHub Copilot, Cursor o equivalente) y comprensión básica de CI/CD.

Se espera que el candidato demuestre habilidades avanzadas en automatización de UI y API, criterio para el uso de asistentes de IA en el flujo de trabajo, y capacidad de justificar decisiones técnicas con criterio propio.

✅ Descripción del Challenge

El candidato deberá implementar una solución de pruebas automatizadas para **Beilox** cubriendo los siguientes bloques:

1️⃣ Pruebas de UI Web

Automatizar la validación de la funcionalidad de **búsqueda** en `centraldepasajes.com.ar` (se usa este sitio como referencia productiva, dado que el proyecto interno aún no tiene un ambiente estable expuesto), asegurando que:

- ✓ Se puedan realizar búsquedas correctamente.
- ✓ Los resultados sean visibles y coherentes con los parámetros ingresados.
- ✓ Se cubran distintos escenarios: búsqueda válida, sin resultados, con datos inválidos, y con volver atrás desde la página de resultados.
- ✓ Se aplique el patrón **Page Object Model** con separación entre selectores, acciones y aserciones.
- ✓ El proyecto esté escrito en **TypeScript con tipado estricto** (sin `any` innecesarios).

2️⃣ Pruebas de API

Implementar pruebas automatizadas sobre la API pública `swapi.tech`, cubriendo:

- `/people`
- `/planets`
- `/films`

Para cada endpoint, el candidato deberá:

- ✓ Implementar al menos tres tests: un caso exitoso (happy path) y uno o dos escenarios de error esperados (400, 404).
- ✓ Validar en cada prueba: **status code**, **estructura del body**, y **tiempo de respuesta razonable**.
- ✓ Validar los schemas de respuesta con **AJV**.
- ✓ Guardar en un archivo JSON el body del happy path para futuras validaciones.

📌 **Importante:** Tanto UI como API deben usarse dentro del mismo proyecto Playwright, organizando los tests con foco en:

- ➔ Escalabilidad
- ➔ Organización
- ➔ Legibilidad
- ➔ Código limpio

3️⃣ Uso de asistente de IA + MCP Servers

El candidato deberá documentar en el README cómo usó su asistente de IA durante el challenge (Claude Code, GitHub Copilot Chat, Cursor o equivalente), incluyendo:

- ✓ Al menos **dos MCP servers** utilizados de forma concreta. Ejemplos válidos:
  - **Playwright MCP** — para explorar la UI y descubrir locators antes de escribir el spec.
  - **Atlassian MCP (Jira)** — para simular la carga de un bug con evidencia adjunta.
  - **Filesystem / Git MCP** — para refactors del repo.
- ✓ Un caso donde **decidió NO usar el asistente de IA** y por qué (criterio del rol).
- ✓ Un **MCP server propuesto por el candidato** que no haya usado durante el challenge pero que considere útil para su día a día como QA Automation — con justificación del caso de uso concreto.

4️⃣ Configuración de CI/CD (GitHub Actions)

El candidato deberá:

- ✓ Configurar un **GitHub Action** que ejecute las pruebas automáticamente **todos los lunes a las 15:00 (GMT-3, Argentina)**.
- ✓ El workflow debe instalar dependencias, correr tests de UI + API + generar el archivo JSON del happy path.
- ✓ Sugerir en el README cómo estructuraría el workflow si la suite creciera significativamente (sharding, matriz por proyecto, cómo pasar secrets, etc). No es necesario implementarlo — se evalúa el criterio.

🎯 Requisitos Técnicos

📌 1. Implementación Técnica

- **UI Web:** Playwright + TypeScript sobre `centraldepasajes.com.ar` cubriendo escenarios diversos.
- **API:** Playwright API Testing con validación de schemas AJV.
- **Estructura del código:** modular, escalable, con separación clara entre tests, page objects, helpers, fixtures y configuración.

📌 2. Buenas Prácticas y Documentación

- Código limpio, reutilizable y tipado.
- Uso de **fixtures propias** (`test.extend`) para inyectar Page Objects, no instanciarlos en cada test.
- Manejo correcto de esperas: usar aserciones auto-esperantes (`expect(...).toBeVisible`) y evitar `waitForTimeout` fijos.
- Manejo de credenciales via `.env` (nunca commiteadas).
- README con:
  - Pasos detallados para ejecutar cada suite (UI, API).
  - Arquitectura del proyecto y decisiones técnicas.
  - Documentación del uso del asistente de IA + MCPs (bloque 3️⃣).
  - Respuestas a las preguntas teóricas (ver abajo).

📌 3. Configuración de CI/CD

- **GitHub Action** que corra UI + API los lunes 15:00 ART.

- Explicación en el README de cómo escalar el workflow ante crecimiento de la suite.

🧠 Preguntas Teóricas

Responder en el README o archivo `.md` aparte, con criterio propio (no copiadas de documentación), en pocas líneas cada una:

1. **Escalabilidad:** Imaginate que ya estamos trabajando juntos y la suite creció a 500 tests. ¿Qué cambios harías o sugerirías en la estructura?
2. **Flakiness:** Si un test es intermitente (a veces pasa, a veces falla) sin cambios en el código, ¿cómo lo diagnosticás y qué opciones considerás antes de marcarlo como skip?
3. **POM:** Si el sitio cambia el selector del botón "Buscar", ¿cuántos archivos del proyecto tendrías que modificar? Justificá.
4. **AI assistant — Uso responsable:** Si tu asistente de IA te genera un test que pasa al primer intento, ¿qué chequeás antes de mergearlo? Contá un caso donde decidiste NO usar el asistente aunque hubieras podido.
5. **Criterio del rol:** ¿Cuál es la responsabilidad de un QA Senior que NO se puede delegar a un asistente de IA? Ejemplos concretos.
6. **CI/CD:** Si los tests fallan cada lunes a las 15:00 por un problema transitorio de red del ambiente, ¿cómo lo manejarías? ¿Retries? ¿Descartar la corrida? ¿Alertar al equipo?

📌 Entrega

- 🔹 **Formato:** Repositorio público en GitHub.
- 🔹 **Tiempo estimado:** 2-3 días.
- 🔹 **Documentación requerida:**
  - README con instrucciones, arquitectura y decisiones técnicas.
  - Respuestas a las 6 preguntas teóricas.
  - Sección específica documentando uso del asistente de IA + MCPs (bloque 3️⃣).
  - Archivo `.json` con el body del happy path de API.

🔹 Criterios de Evaluación

- Correctitud y funcionalidad de las pruebas (UI, API).
- **Calidad del código y arquitectura** — tipado estricto, POM aplicado, sin duplicación innecesaria.
- **Aplicación de buenas prácticas de automatización** — esperas correctas, aislamiento entre tests, manejo de credenciales.
- **Uso responsable del asistente de IA + MCPs** — que quede claro cómo lo usó y dónde decidió no usarlo. Se valora criterio, no cantidad.
- Implementación del GitHub Action correctamente.
- Claridad y detalle en la documentación.
- **Proactividad y criterio propio** — reconocer limitaciones, proponer mejoras, justificar decisiones.
- **Respuestas a las preguntas teóricas** — se valoran respuestas concisas y con criterio; no se busca la respuesta perfecta sino la comprensión real.

🚀 Tecnologías Requeridas

- ✅ **Playwright con TypeScript** (obligatorio TypeScript, no JavaScript)
- ✅ **AJV** para validación de schemas en pruebas de API
- ✅ **Asistente de IA compatible con MCP** (Claude Code, GitHub Copilot Chat, Cursor o equivalente)
- ✅ **GitHub Actions** para CI/CD

🔥 Notas Finales

- Este challenge representa un escenario real de trabajo, evaluando habilidades clave en automatización web y API, uso de herramientas de IA con criterio, y capacidad de comunicar decisiones técnicas.
- **No buscamos una solución perfecta.** Buscamos una implementación que demuestre conocimiento, criterio y capacidad de justificar decisiones.
- Se valorará especialmente:
  - La **organización y escalabilidad** de la solución.
  - El **criterio para usar (o no usar)** herramientas de IA.
  - La **honestidad al reconocer trade-offs** — qué se dejó afuera y por qué.

¡Éxitos con el challenge!

Documento de uso interno — Equipo de QA Automation · Julio 2026